

Documento de Arquitectura del Sistema (DAS)

Título del Proyecto: [S.G.A Óptica - Implementación de un Sistema de Gestión de Citas Online Para Óptica]

Versión: 1.0

Fecha: [11/2024]

Elaborado por:

S.G.A Óptica

Tabla de Contenido

1. Introducción
2. Problema
3. Solución Propuesta
4. Objetivos
 - 4.1. Objetivo general
 - 4.2. Objetivos específicos
5. Propuesta Arquitectónica
 - 5.1. Definición y Estilo de la Arquitectura
 - 5.2. Perspectiva Lógica (Vista de Casos de Uso)
 - 5.3. Perspectiva del Desarrollo (Despliegue e Implementación)
 - 5.4. Requisitos de Calidad
6. Definiciones Técnicas
 - 6.1. Lenguajes de Programación y Frameworks
 - 6.2. Lenguaje de Programación de Base de Datos
 - 6.3. Extensiones y Librerías Requeridas
7. Requisitos de Hardware y Despliegue
8. Consideraciones

1. Introducción

La óptica Unidad Visual es un establecimiento de salud visual ubicado en Bogotá D.C. que ofrece servicios de optometría, venta de monturas, lentes y accesorios ópticos. En el contexto colombiano, la demanda de servicios visuales ha crecido significativamente debido al envejecimiento de la población, el uso intensivo de dispositivos electrónicos y la limitada oferta de profesionales: Colombia cuenta con apenas 10 facultades de optometría para más de 50 millones de habitantes.

El presente Documento de Arquitectura del Sistema (DAS) define la estructura técnica del Sistema de Gestión de Agendamiento (SGA), desarrollado bajo los lineamientos del SENA y la norma ANSI/IEEE 830-1998. El DAS describe los estilos arquitectónicos seleccionados, las perspectivas lógicas y de despliegue, las decisiones tecnológicas y los requisitos de calidad que guiarán la implementación.

2. Problema

Actualmente, la óptica Unidad Visual gestiona sus citas de forma telefónica o presencial, lo que genera las siguientes problemáticas:

- Dependencia total del tiempo y disponibilidad del personal administrativo para registrar citas.
- Errores frecuentes de duplicación, pérdida o desorganización de información de citas.
- Ausencia de recordatorios automáticos, incrementando cancelaciones e inasistencias de último momento.
- Experiencia deficiente para el usuario: largas esperas, llamadas sin respuesta y desplazamientos innecesarios.
- Falta de trazabilidad del historial clínico de los pacientes y de sus fórmulas ópticas.
- Gestión manual del inventario de productos (monturas, lentes, accesorios), propensa a inconsistencias.
- Ausencia de mecanismos de reportes o análisis de la operación del negocio.

Esta situación coloca a Unidad Visual en desventaja competitiva frente a ópticas que ya operan con plataformas digitales modernas.

3. Solución Propuesta

Se propone desarrollar e implementar una aplicación web (SGA Óptica) que centralice y digitalice la operación de Unidad Visual. La solución tecnológica contempla:

- Una plataforma web accesible desde computadores y dispositivos móviles (diseño responsive).
- Módulo de agendamiento en línea con disponibilidad en tiempo real de los optómetras.
- Envío automático de notificaciones y recordatorios por correo electrónico o SMS.
- Gestión de historial clínico y fórmulas ópticas de cada paciente.
- Módulo de inventario y tienda en línea para compra de monturas y lentes.
- Panel de administración con generación de reportes y gestión de usuarios.

La arquitectura se basa en un modelo de tres capas (Presentación, Lógica de Negocio y Datos), soportado por tecnologías web estándar (Node.js, React, HTML, Bootstrap, JavaScript, MySQL), garantizando escalabilidad, seguridad y facilidad de mantenimiento.

4. Objetivos

4.1. Objetivo general

Desarrollar e implementar un sistema de gestión de citas en línea para la óptica Unidad Visual en Bogotá D.C., que permita a los clientes agendar, modificar y cancelar citas de manera autónoma y eficiente, con visualización en tiempo real de la disponibilidad de los optómetras y gestión integral de los procesos internos del negocio.

4.2. Objetivos específicos

- Desarrollar y automatizar una interfaz en línea para el agendamiento de citas, permitiendo reservas rápidas y sencillas desde el sitio web de la óptica.
- Vincular el sistema de citas con el calendario de los optómetras para asegurar disponibilidad en tiempo real.
- Implementar notificaciones automáticas por correo electrónico o SMS para recordatorios de citas y reducir inasistencias.
- Permitir la modificación o cancelación de citas a través de la plataforma en línea, mejorando la flexibilidad para los usuarios.
- Habilitar la compra de artículos (monturas, lentes, accesorios) a través de la página web.
- Permitir a administradores y empleados consultar y gestionar el inventario disponible.
- Integrar una base de datos segura que registre el historial clínico y de citas de cada cliente.
- Diseñar un esquema arquitectónico documentado que sirva de referencia para la implementación y futuras ampliaciones.
- Realizar pruebas y evaluaciones periódicas del sistema para asegurar su correcto funcionamiento.

5. Propuesta Arquitectónica

El objeto de esta sección es sustentar la solución propuesta desde la arquitectura técnica, sirviendo como referencia para el diseño y análisis de la implementación del SGA.

5.1. Definición y Estilo de la Arquitectura

- **Estilo Principal: Arquitectura Monolítica en Capas (N-Tier)**
Se adopta una arquitectura monolítica de 3 capas, apropiada para el alcance y equipo del proyecto:

Capa	Responsabilidad	Tecnología
Presentación (Frontend)	Interfaz de usuario, formularios, vistas responsive	HTML5, Bootstrap, React
Lógica de Negocio (Backend)	Reglas de negocio, APIs REST, autenticación, notificaciones	Node.js (JavaScript)
Datos (Base de Datos)	Persistencia de datos, consultas, transacciones	MySQL / PhpMyAdmin

- **Patrones de Diseño:**
 - MVC (Modelo-Vista-Controlador): separación clara entre la lógica de negocio, la presentación y el acceso a datos.
 - Patrón de Repositorio: abstracción del acceso a la base de datos para facilitar el mantenimiento y las pruebas.
 - API REST: comunicación entre el frontend (React/Bootstrap) y el backend (Node.js) mediante Axios.
- **Vistas de Alto Nivel:** El sistema se estructura en tres capas diferenciadas. La capa de Presentación es la interfaz con la que interactúa el usuario a través del navegador, construida con HTML/Bootstrap y React. La capa de Lógica de Negocio centraliza las reglas del sistema y expone una API REST desarrollada en Node.js, gestionando autenticación, notificaciones y procesos de negocio. La capa de Datos almacena y persiste toda la información del sistema en MySQL, accesible únicamente desde el backend. Esta separación aporta mayor flexibilidad, seguridad y facilita el mantenimiento independiente de cada componente.

5.2. Perspectiva Lógica (Vista de Casos de Uso)

Actores: Administrador, Cliente, Usuario, Empleado

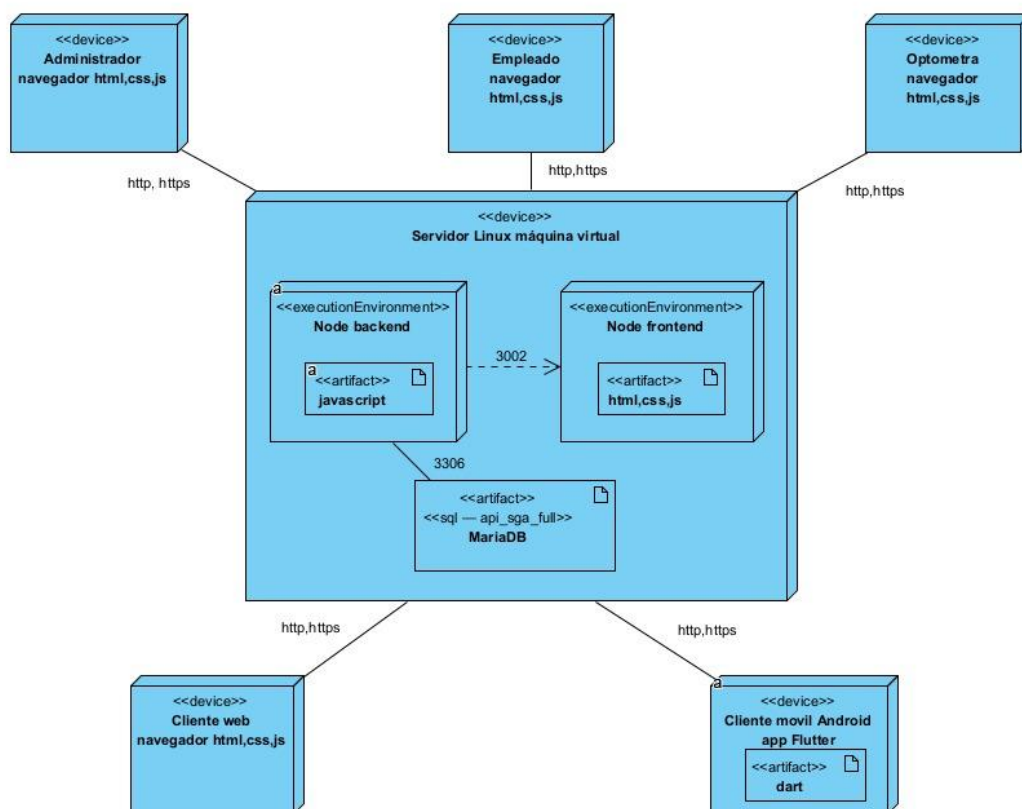
Casos de Uso Principales:

- RF001 — Registro del cliente
- RF002 — Inicio de sesión y autenticación
- RF003 — Gestión de citas médicas (agendar, modificar, cancelar)
- RF004 — Visualización de productos
- RF005 — Subida y consulta de fórmula visual
- RF006 — Gestión de inventario (Administrador)
- RF007 — Consulta del historial de visitas y exámenes
- RF008 — Gestión de usuarios (Administrador)
- RF009 — Recuperación de contraseña
- RF011 — Envío de recordatorios automáticos de citas
- RF013 — Proceso de compra de gafas/monturas
- RF014 — Generación de reportes por fecha
- RF015 — Registro manual de pagos físicos (Empleado)
- RF016 — Gestión de optómetras (Administrador)
- RF017 — Consulta del historial óptico por el optómetra

Diagrama de Casos de Uso

5.3. Perspectiva del Desarrollo (Despliegue e Implementación)

- **Diagrama de Despliegue:** El diagrama UML de despliegue muestra los nodos físicos del sistema: cuatro dispositivos cliente (Administrador, Óptometra, Empleado y Cliente) que se conectan vía HTTP/HTTPS al servidor central, el cual contiene los entornos de ejecución Node backend (Java/Node.js, puerto 543) y Node frontend (HTML, CSS, JS). El servidor de base de datos corre Linux con MySQL en el puerto 3306. La imagen a continuación es el diagrama UML oficial del proyecto.



- **Detalle del desarrollo (Diagrama Físico Relacional):**

El modelo relacional contiene lo siguiente:

- rol = (id, name)
- user = (user_id, user_user, user_password, role_id, createdAt, updatedAt)
- user_entity = (id, user_id, first_name, last_name, phone, address)
- customer = (customer_id, id_user, id_doc_type, documentNumber, firstName, secondName, firstLastName, secondLastName, phoneNumber, email, createdAt, updatedAt)

- document_type = (id_doc_type, type_document, document_name, status)
- optometrist = (id, documentNumber, email, first_name, second_name, first_last_name, second_last_name, phone_number, id_doc_type, id_user, professional_card_code)
- exam_type = (id, name, description)
- appointment = (appointment_id, date, time, status, customer_id, exam_type_id, createdAt, updatedAt, optometrist_id)
- notification = (notification_id, type, subject, message, sent_at, status, customer_id, appointment_id)
- sale = (id, sale_date, number_bill, total, customer_id, payment_type_id)
- product = (id, name_product, stock, unit_price, status, category_id, image)
- sale_product = (id, quantity, sell_price, id_sale, id_product)
- payment_type = (id, name)
- category = (category_id, category_name)

- **Diagrama de Base de Datos:**

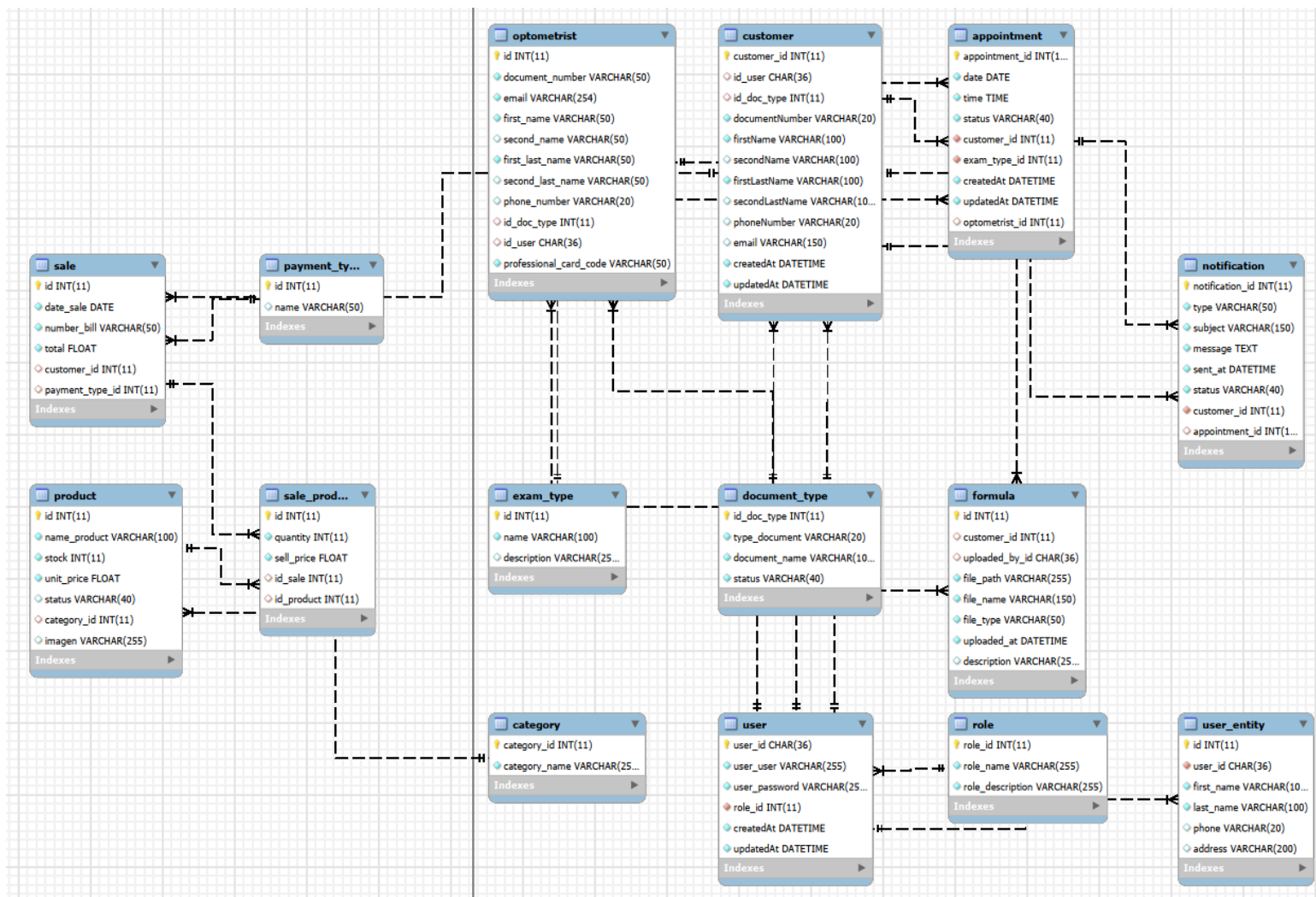
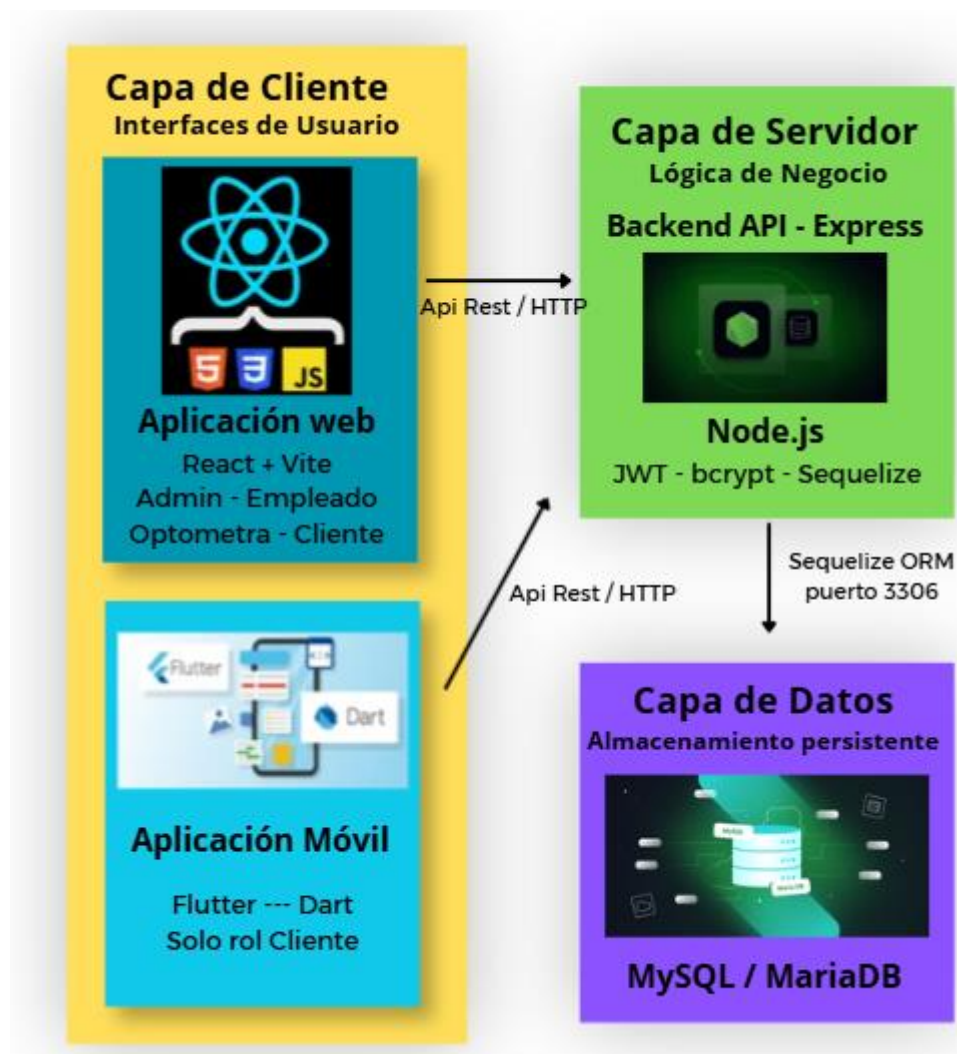


Diagrama de Clases

- **Diagrama de Arquitectura:** El siguiente diagrama muestra la arquitectura del sistema SGA Óptica con sus tres capas: Presentación (navegadores web de los 4 roles: Administrador, Optómetra, Empleado y Cliente), Lógica de Negocio (NGINX como proxy inverso, Node Frontend con Vite+React, Node Backend con Express.js+JWT+Sequelize, y Servicio de Email vía SMTP/Nodemailer) y Datos (MySQL/MariaDB con 15 tablas relacionales, gestionado con PhpMyAdmin). La comunicación entre capas usa HTTPS (HTTP/S) y Axios para la API REST en /api/v1.



5.4. Requisitos de Calidad

La arquitectura de 3 capas con Node.js, MySQL y NGINX contribuye directamente a los atributos de calidad definidos en el SRS:

Atributo	Mecanismo Arquitectonico	Requisito SRS
Seguridad	Autenticación JWT, cifrado SSL/TLS (HTTPS), control de acceso por roles (RBAC), cifrado de contraseñas con bcrypt	RNF001, RNF008
Rendimiento	NGINX como proxy inverso y servidor de archivos estáticos, índices en MySQL, consultas optimizadas	RNF003, RNF005
Escalabilidad	Arquitectura de 3 niveles permite escalar horizontalmente el backend; MySQL soporta réplica de lectura	RNF006, RNF009
Disponibilidad	Estrategia de backups periódicos en MySQL, entorno de staging para validar actualizaciones, certificado SSL activo	RNF004, RNF006
Accesibilidad	Diseño responsive con Bootstrap, compatible con móviles y escritorio	RNF010
Escalabilidad	Despliegue sobre Linux Debian 13, Node.js multiplataforma, MariaDB	RNF (Portabilidad)

6. Definiciones Técnicas

6.1. Lenguajes de Programación y Frameworks

- **Backend:** (JavaScript / Node.js).
- **Frontend:** (Html / Bootstrap + React).
- **Comunicación:** (Axios HTTP Client)

6.2. Lenguaje de Programación de Base de Datos

- **Motor:** (MySQL).
- **Herramienta de Gestión:** (PhpMyAdmin).

6.3. Extensiones y Librerías Requeridas

- React (Frontend)
- Axios (Conexión de Apis, Frontend y Backend)
- Error Lens (IDE VS Code)
- Express js (Backend)
- bcrypt (Backend)
- jsonwebtoken (JWT, Backend)
- mysql2 (Backend)

7. Requisitos de Hardware y Despliegue

Componente	Mínimo Requerido
Servidor de Aplicaciones	4GB RAM, 2 VCPU, 20 GB Almacenamiento
Servidor de Base de Datos	2GB RAM, 10 GB Almacenamiento
Sistema Operativo	Debian 13 Linux o Windows
Servidor Web/Proxy	NGINX

8. Consideraciones

- Requisitos de mantenimiento y actualización de software: se deberán ejecutar revisiones periódicas de dependencias (Node.js, librerías npm, MySQL) y aplicar parches de seguridad en los tiempos definidos por el equipo de desarrollo.
- Estrategia de *backup* y controles de seguridad: se debe implementar un backup automático diario de la base de datos MySQL, con cifrado de datos sensibles (historiales clínicos, fórmulas ópticas) en tránsito mediante SSL/TLS y en reposo.
- Necesidad de entornos de prueba (staging o gemelo) para validación de actualizaciones: antes de cualquier despliegue en producción, los cambios deben validarse en un entorno idéntico al productivo para evitar interrupciones del servicio.
- Gestión del certificado de seguridad SSL: el certificado SSL debe renovarse antes de su vencimiento; se recomienda configurar la renovación automática y establecer alertas de monitoreo.